Browser lifetime + stop rule: test results

- Branch: `fix/browser-lifetime-and-stop-rule`, base `b676c2d`
- Fix commit: `f2288ce`. Tests and harness commit: `caaa191`. Both are pushed to origin.
- Run on 2026-10-12, on this VM (2 vCPU). No Railway run, no paid run, no real LLM calls.
- Evidence files: `evidence/browser-lifetime-candidate/` (repro JSON and logs) and `evidence/regression-caaa191/` (suite logs, tsc output and integration JSON).

1. Before/after repro (server/scripts/browser-lifetime-repro.ts)

Setup. `WORKER_ISOLATION_ENABLED=true`, worker concurrency 4, `MAX_CONCURRENT_BROWSER=1`, 12 jobs, 1 document per child, 20 s synthetic scoring hold after the fetch. Each document is a synthetic PDF with its own text marker. Each run checks that every document was requested, extracted with its marker, and never skipped as BrowserUnavailable.

How the runs were made. Each side ran in its own git worktree: `/tmp/wt-base` at `b676c2d` and `/tmp/wt-cand` at `f2288ce`. Both used the same harness files, copied from `caaa191`, so only `processor.ts` / `worker.ts` differ. Runner: `evidence/browser-lifetime-candidate/wt-run.sh`. The TasksMax runs used `systemd-run --scope -p TasksMax=400`, i.e. a cgroup pids limit of 400.

Metric	Base, unlimited	Candidate, unlimited	Base, TasksMax=400	Candidate, TasksMax=400
Peak live browsers (distinct profiles)	**4**	**1**	**4**	**1**
Live-browser bound (≤ permits) held	no	yes	no	yes
Peak tasks (threads)	550	205	367	180
Peak processes	42	19	29	16
Peak summed RSS (MB, upper bound)	4,941	2,048	3,398	1,954
Jobs ok / failed	12 / 0	12 / 0	**8 / 4**	12 / 0
Docs extracted with marker	12/12	12/12	**8/12**	12/12
Chromium launches	12	12	15	12
“Failed to launch the browser process” lines	0	0	6	0
Wall time	126.3 s	127.4 s	**501.9 s**	127.6 s
Admission wait, mean / p95	6.1 s / 24.7 s	6.4 s / 24.5 s	93.2 s / 136.4 s	6.5 s / 24.9 s
Permit hold, mean / p95	8.3 s / 8.6 s	8.4 s / 8.6 s	39.6 s / 126.5 s	8.4 s / 8.6 s
Per-document fetch, mean	14.4 s	14.9 s	132.8 s	14.9 s
Final broker permits active	0	0	0	0
Exit	0	0	0	0

Reading the table

- **Unlimited.** The base keeps one Chromium alive per child until that child exits. That gives 4 live browsers with 1 permit. The candidate never has more than 1. Threads fall by about 63% (550 → 205) and summed RSS by about 59%. Wall time, admission wait and permit hold are unchanged: closing the browser before releasing the permit adds no measurable latency here (permit hold 8.3 s vs 8.4 s).
- **TasksMax=400.** This cgroup pids limit makes the base fail the same way production does, as browser-launch failures:
- Puppeteer printed 6 “Failed to launch the browser process … Code: null” lines and 3 “Network.enable timed out” lines, and one fetch hit “Timed out after waiting 30000ms”.
- 4/12 jobs ended with `no document text`. Their PDFs were never requested (`neverRequested` = job-1002/1005/1008/1011). Admission wait rose to a mean of 93 s, and the run took 3.9× as long.
- The candidate under the same limit stays at 1 browser, 180 tasks and 12/12 jobs, the same as its unlimited run.
- **Caveat.** The production SIGABRT signature was **not** reproduced locally. What reproduced is the launch-failure / starvation mode under a task cap. The harness labels a failed fetch “fetch ok … chars=0 marker=false” in the child log line. The summary `extraction` block is the authoritative per-document check.

The earlier repro evidence in `evidence/browser-lifetime/` was made in the main checkout before the worktree method, with the old counter. It is superseded by this table.

2. New tests

Suite	Result	What it proves
`server/lib/browser-close-order.test.ts`	**4/4 pass** (56 s)	On the success, fetch-failure (direct + pdfRecovery), cancellation (in-flight / post-acquire refusal / pre-acquire refusal) and child-death paths, the child’s Chromium process group is gone **before** the browser permit returns to the parent broker. On the child-death path, the parent reaps before it releases.
`server/lib/batch-stop-rule.test.ts`	**11/12 pass, 1 skipped**	The skipped test is optional: SQL against a local PG. See §4.

Mutation check (from the earlier session, same code). Releasing the permit before the reap check makes the close-order tests fail: more than one live Chromium at permit return. So the ordering is what the tests detect, not an accident.

3. Regression suites

Each suite runs with `DATABASE_URL=postgres://stub:stub@127.0.0.1:1/stub npx tsx server/lib/<suite>.test.ts` at `caaa191`. The 11 stub suites ran in parallel, then browser-close-order and job-isolation ran together after the repro runs had finished.

Suite	Result	Exit
job-isolation	29/29	0
lifecycle-fence.unit	42/42	0
scoring-cancellation.unit	60/60	0
credit-breaker	22/22	0
llm-usage	20/20	0
adaptive-concurrency	13/13	0
runtime-diag.unit	21/21	0
reconcile-resurrect	15/15	0
reliability	PASS (20 acceptance checks)	0
provider-resilience	112/112	0
transport-abort.unit	55/55	0
browser-close-order (new)	4/4	0
batch-stop-rule (new)	11/11 + 1 skip	0

These counts match the previous branch baseline (`TEST_RESULTS` for `fix/worker-isolation`) for every existing suite.

Integration suite (opt-in, real Postgres/Redis/BullMQ/Chromium)

`DATABASE_URL=postgres://ciq:ciq@127.0.0.1:55432/ciq REDIS_URL=redis://127.0.0.1:56379 INT_REQUIRED=1 npx tsx server/lib/job-isolation.integration.test.ts` (local docker `postgres:16-alpine` + `redis:7-alpine`, concurrency 2):

Run	Result	Peak Chromium under job children
Base `b676c2d` (worktree)	PASS 11/11	18 procs, 2,398 MB
Candidate run 1	**FAIL 2/11**	10 procs, 1,381 MB
Candidate run 2	PASS 11/11	10 procs, 1,377 MB

Candidate run 1 failures

- “2 isolated job children ran concurrently (max seen 3)”.
- “no descendants after completion (1, chromium 0)”.

In both, the extra process was one direct child of the test process, at a steady ~14 MB. It was present from the first sample to after `stopWorker()`, and it was not Chromium.

Likely cause (hypothesis, not proven). The extra process is tsx’s `esbuild --service` helper, started after the test took its baseline snapshot.

- In the base run, a `ps --ppid` snapshot showed exactly this process (esbuild, ~15 MB) as a direct child. It was present at baseline there, so it was excluded.
- In run 1 the log says “baseline descendants excluded: none”.

The test’s own accounting is timing-sensitive: it counts every direct child as a job child. Product code is not involved: all job, batch, score, permit, orphan and quarantine checks passed in run 1. Run 2 passed every check. Not yet done: a run that records the process name of the extra child in run 1’s situation.

The fix also shows in the integration run: peak Chromium under 2 concurrent children fell from 18 to 10 processes (one browser at a time instead of one per child). Summed RSS fell from 2,398 to ~1,380 MB.

4. Stop rule S4 (`BATCH_STOP_RULE_FAILED_JOBS`)

The rule is off when the variable is unset, 0 or junk (test 1). With the variable unset, the worker’s retry path is unchanged from the legacy one: the rule never counts and never cancels (test 9).

Scenario (batch-1276 shape)	Outcome
Rule off	13 jobs exhaust retries (39 failed attempts), as happened in batch 1276
`BATCH_STOP_RULE_FAILED_JOBS=2`	Batch cancelled at the **2nd** failed job, before any job exhausts retries
N=3	The 3rd failed job trips the rule. The batch is cancelled, a snapshot saved and the queue purged, with no further retries or claims

Other behaviour:

- Cancellations and provider-quota pauses do not count as S4 failures.
- If the failure count errors, the rule fails open to the legacy retry path.
- The cancel sequence runs in this order: flag, then purge, then snapshot, then PG cancel. Purge and snapshot errors are not fatal. A cancel-sequence error still suppresses the retry.

5. tsc

`npx tsc --noEmit -p .` gives 62 errors on the candidate and 62 on the base worktree. The sorted error lists match once line/column numbers are removed (`diff` empty). The only processor.ts entry is

the existing `pdf-parse` TS7016, which is present on the base too. The branch adds no new type errors.

6. Not verified

- **Production SIGABRT.** Not reproduced locally. The link between the browser-lifetime defect and the production failures is inferred from:
- the local launch-failure mode under TasksMax=400;
- the batch 1276 URL reconciliation: Southern lost 37% of its URLs, all with browser-launch failures (see `evidence/batch1276-url-reconciliation.md`).
- **Not tested on Railway** or at production settings. `WORKER_ISOLATION_ENABLED` stays **off** by default, so the fix is inactive in production until isolation is enabled. The stop rule is inactive until `BATCH_STOP_RULE_FAILED_JOBS` is set.
- **No real web fetches or LLM providers.** Repro and integration use locally served PDFs and stubbed providers.
- **Unconfirmed-close path.** The permit is withheld and the browser circuit opened. This is covered by unit logic only; it did not occur in any run (quarantined 0).
- **Stop-rule SQL** (`COUNT_BATCH_FAILURES_SQL`) was not run against a real PG in this session (optional test skipped).